

AI BlogNest API Project Report

Technology Track AI Augmented Backend Development

Project Title AI BlogNest API

This report documents the API workflow shown in the supplied screenshots, from account registration and JWT login through blog creation and Gemini assisted content generation. The final screenshots show the model setting and a summarization request. Sensitive values visible in the source images have been covered in this report.

Project overview

AI BlogNest API is a Node.js and Express backend that connects to MongoDB Atlas for blog data and uses JWT authentication to protect requests. A Google Gemini integration supports blog generation and text summarization. Thunder Client is used to exercise the routes shown here.

Observed workflow

Register a user. POST /api/auth/register returned 201 Created.

Log in. POST /api/auth/login returned 200 OK and a JWT token.

Create a blog. POST /api/blogs returned 201 Created after the route, request body and bearer token were corrected.

Generate a blog. POST /api/ai/generate-blog returned 201 Created with generated React.js blog content.

Update AI model. The environment setting was changed to GEMINI_MODEL=gemini-3.5-flash-lite.

Summarize text. POST /api/ai/summarize returned 200 OK. For the short input shown, the AI asked for more content rather than returning a useful summary.

1 Environment and project setup

The project includes route, controller, middleware, model and service folders. The environment provides connection and AI settings. Credentials have been masked in the screenshot.

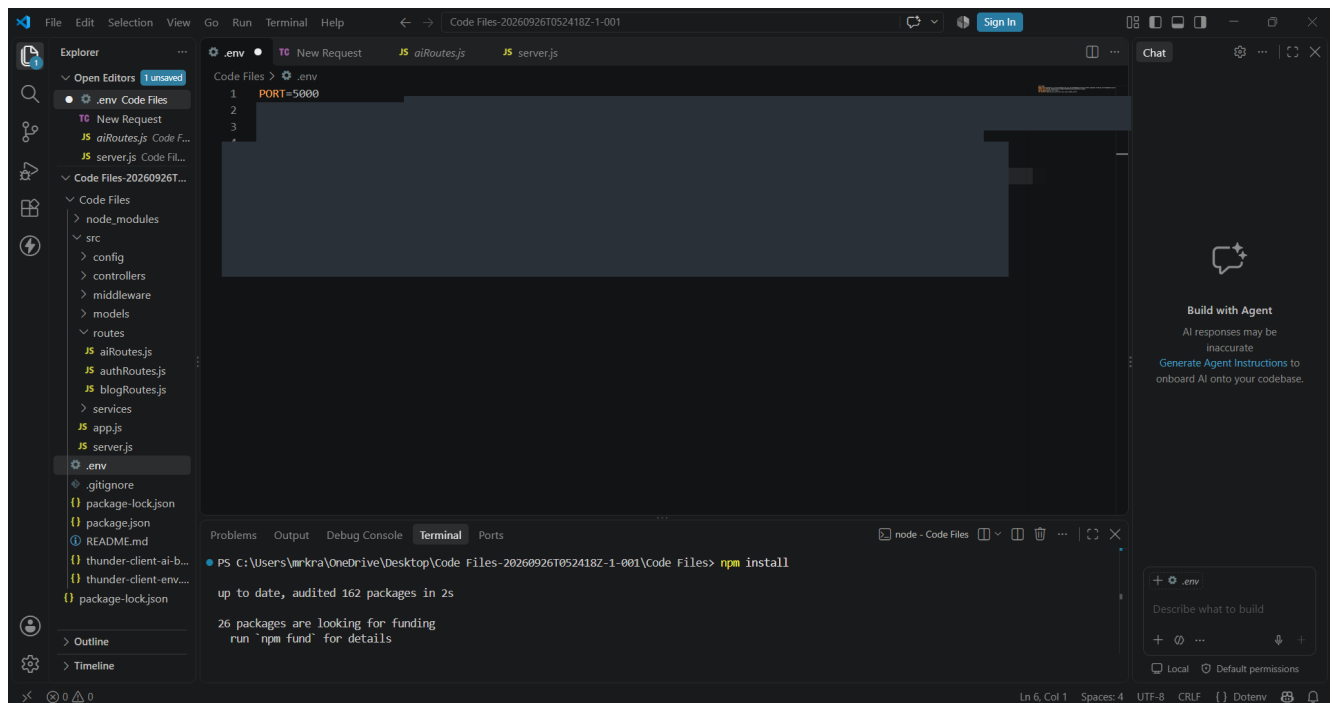


Image 1 Environment and project setup

2 Register a user

The registration endpoint returns 201 Created, confirming that a user record was created.

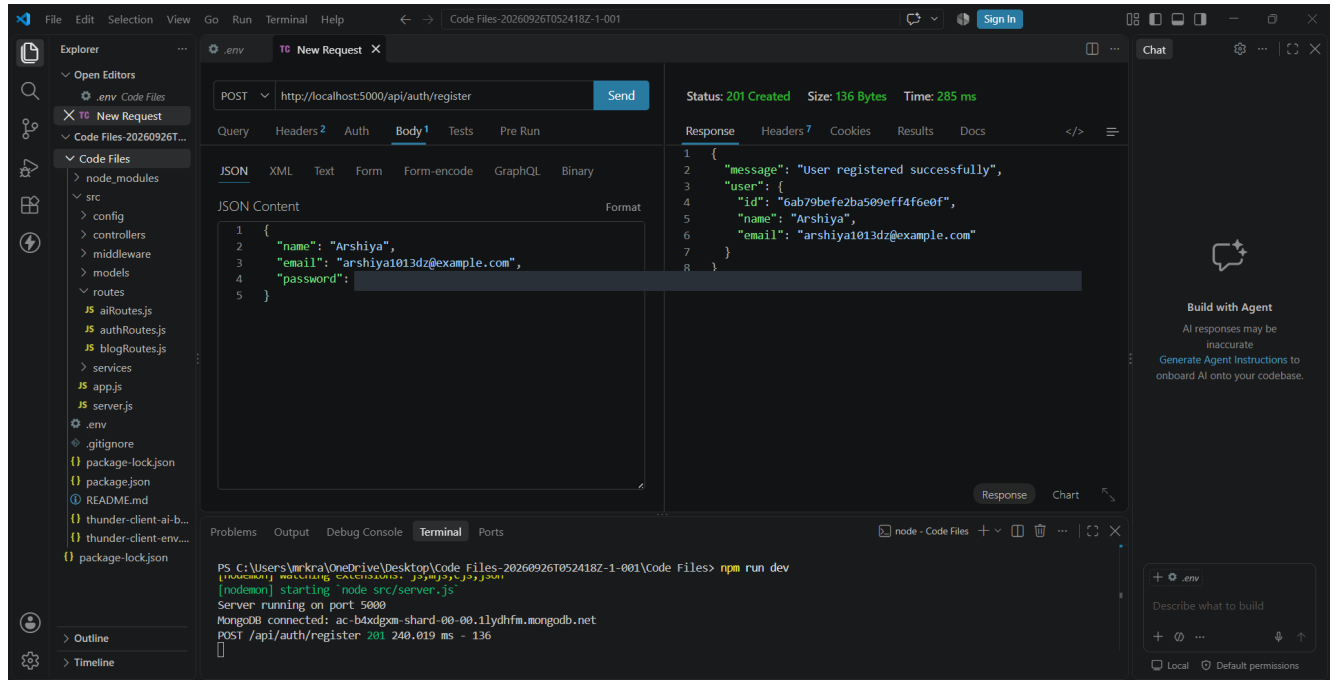


Image 2 Register a user

3 Log in and receive a token

The login endpoint returns 200 OK and a JWT. The token is covered because it grants access to protected routes.

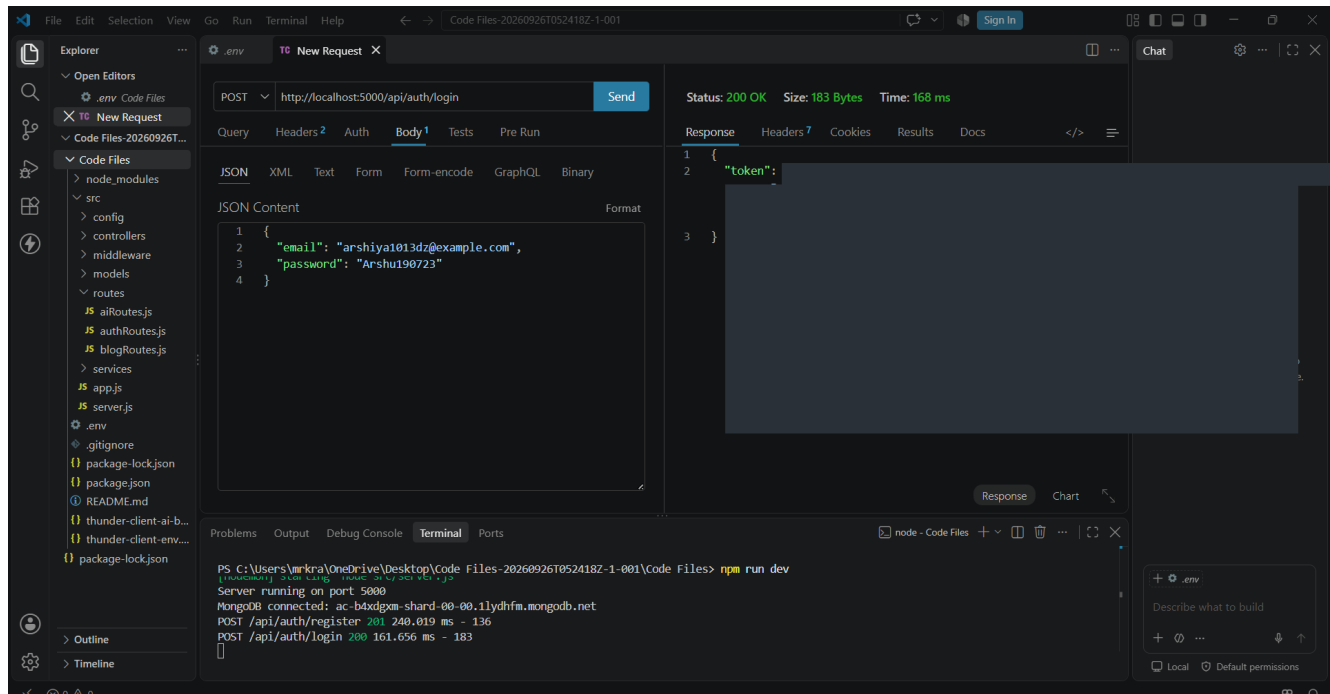


Image 3 Log in and receive a token

4 Configure bearer authentication

Thunder Client sends the JWT through the Authorization header when calling a protected endpoint. The header value is masked.

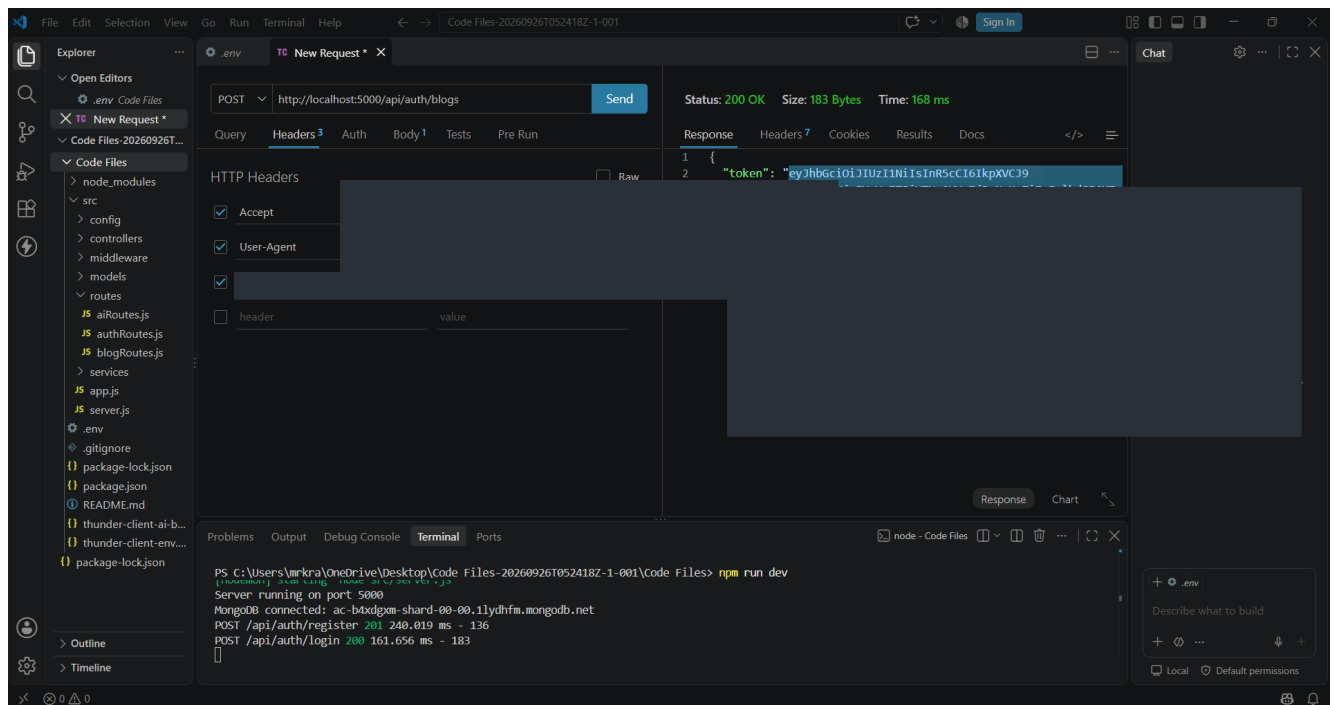


Image 4 Configure bearer authentication

5 Prepare the blog payload

The request body contains title, content and category fields. The blog route is POST /api/blogs.

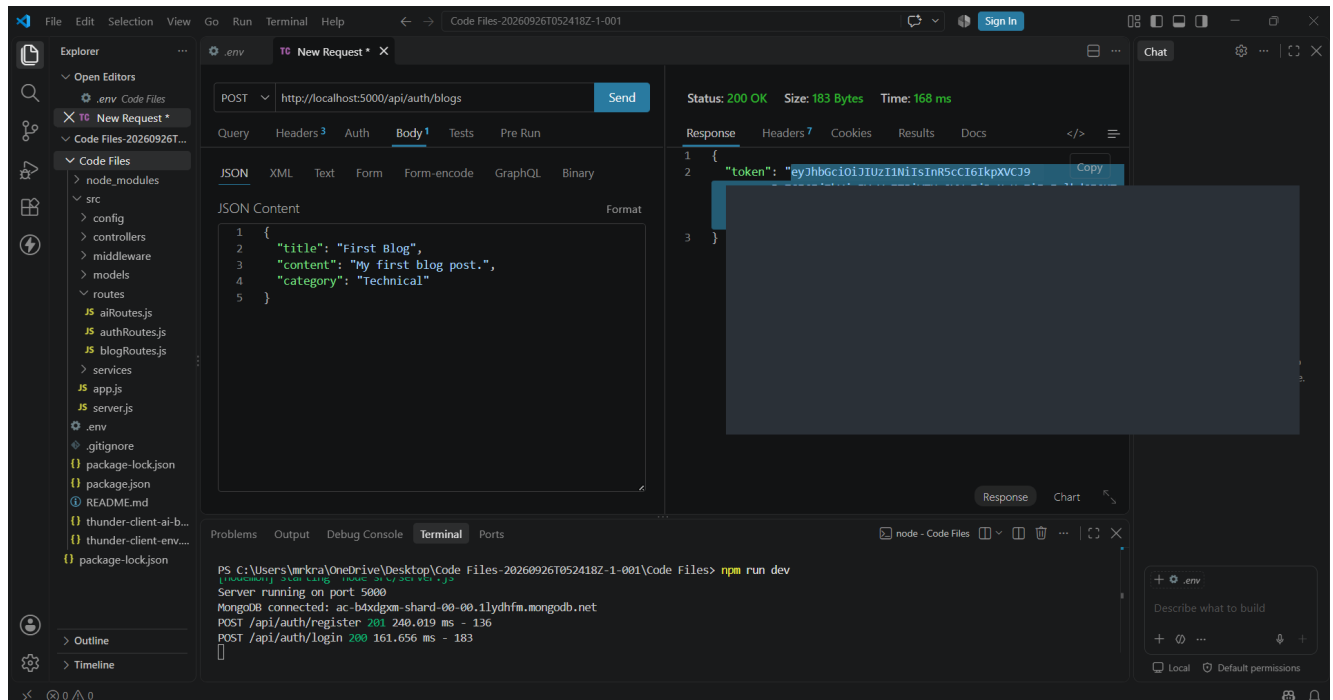


Image 5 Prepare the blog payload

6 Create the first blog

The corrected request returns 201 Created with the blog record in the response.

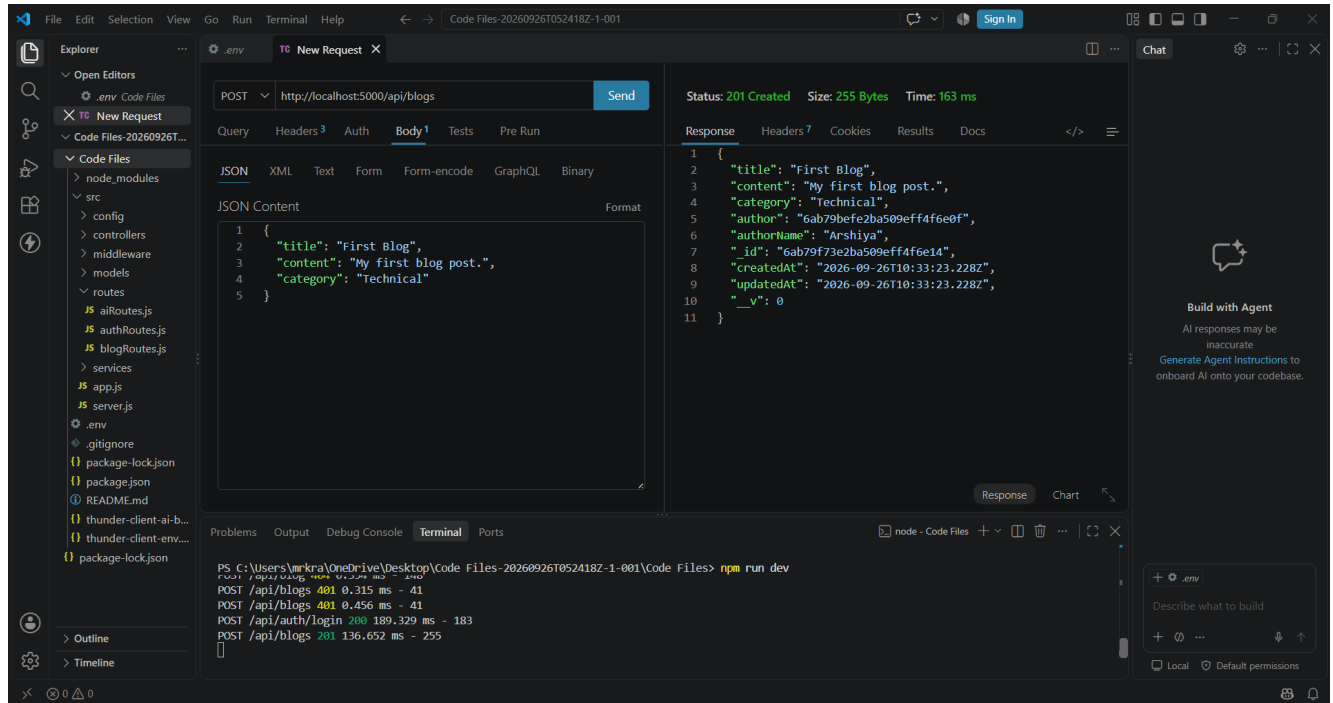


Image 6 Create the first blog

7 Generate blog content with AI

The generate blog route accepts a topic and category and returns 201 Created with generated blog content.

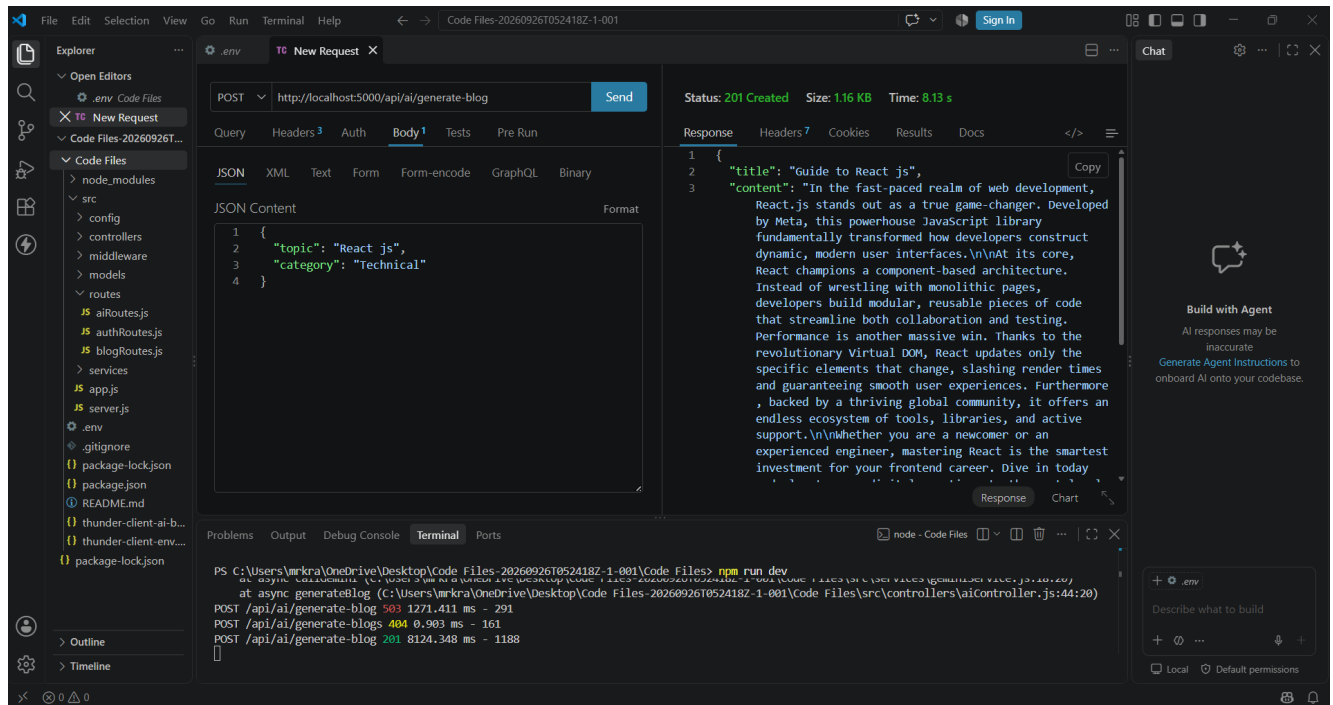


Image 7 Generate blog content with AI

8 Update the Gemini model setting

The AI model configuration was updated before the summarization request. Credentials and secret values have been masked.

Model setting applied after image 7: GEMINI_MODEL=gemini-3.5-flash-lite

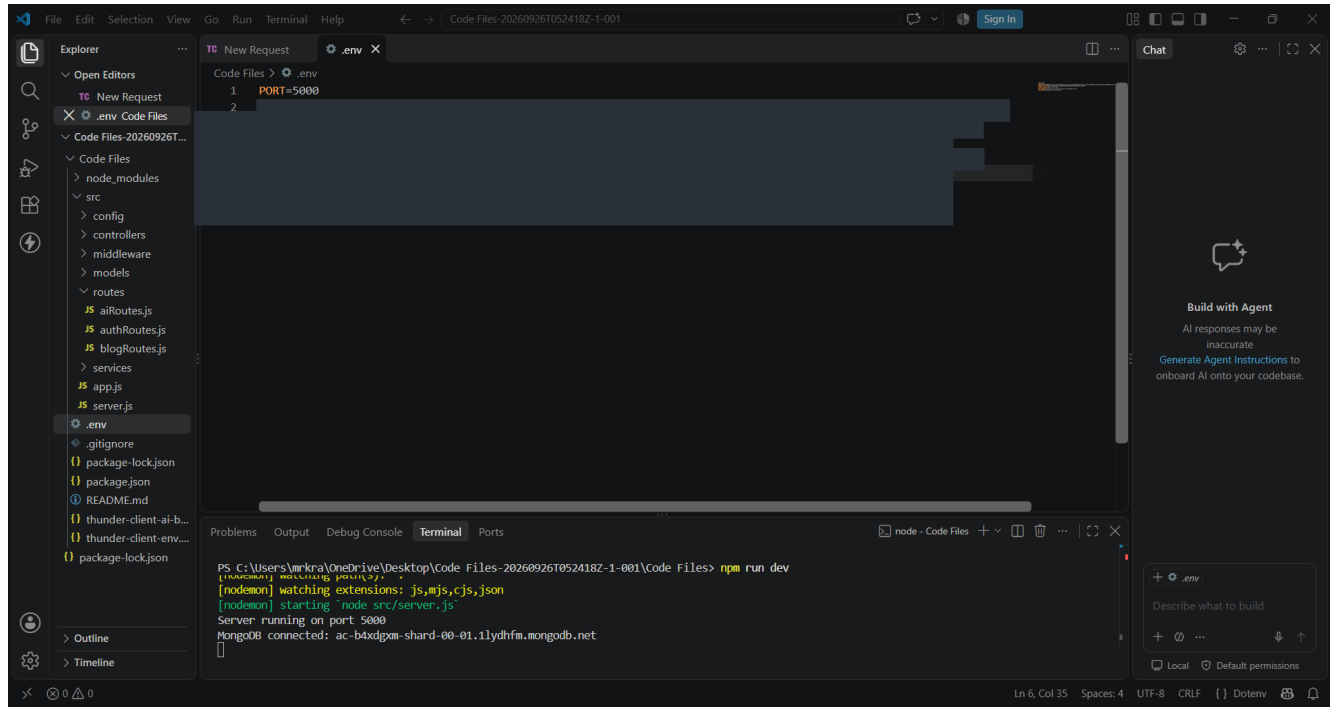


Image 8 Update the Gemini model setting

9 Call the summarization endpoint

A POST request to `/api/ai/summarize` uses a content field. The screenshot shows a 200 OK response, but the brief prompt did not yield a substantive summary.

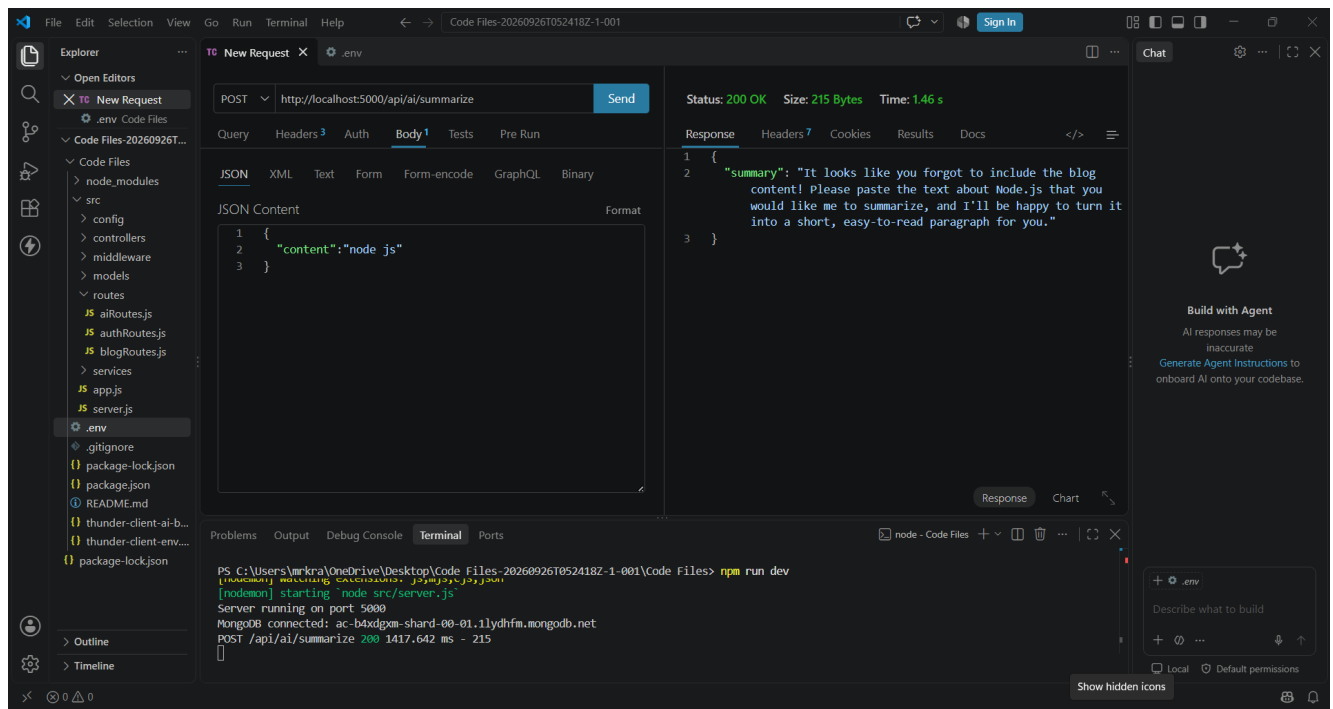


Image 9 Call the summarization endpoint

10 Inspect the blog list request

The final image shows GET /api/blogs returning 200 OK and a JSON list of blog records, including generated React.js content.

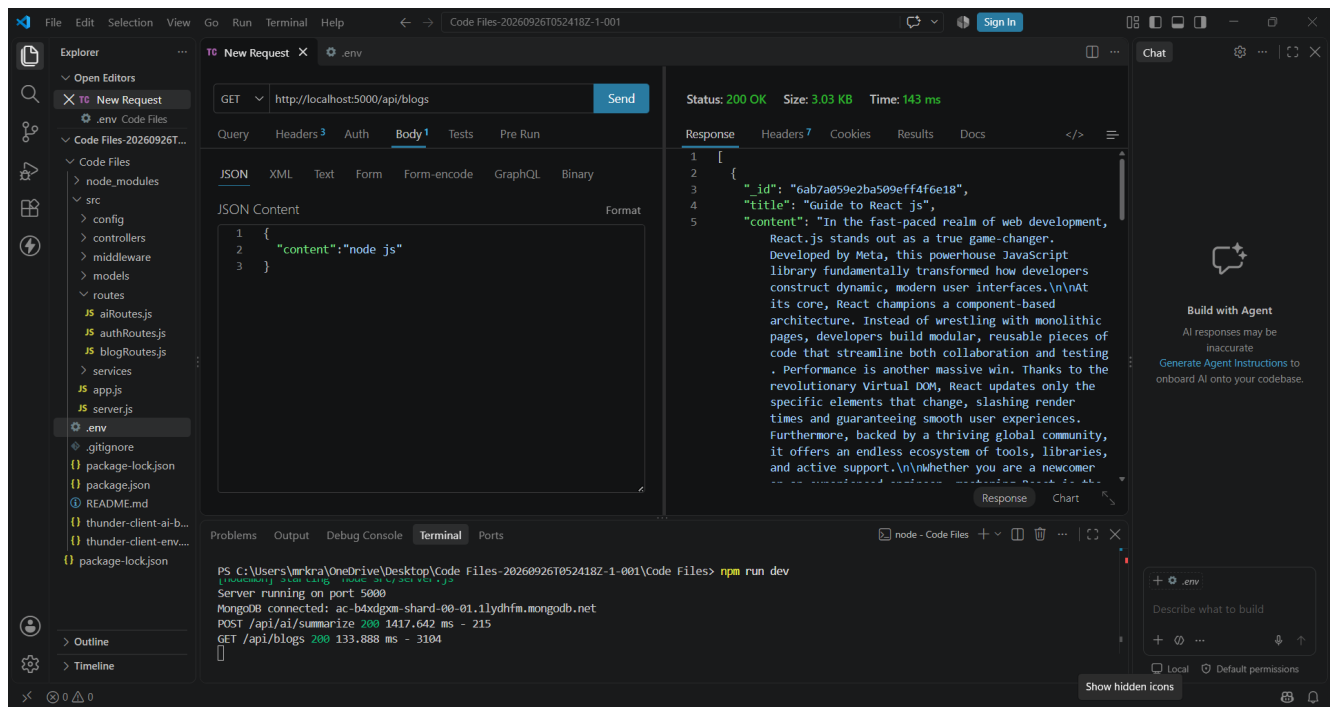


Image 10 Inspect the blog list request

Conclusion

The supplied evidence confirms registration, login, authenticated blog creation, AI blog generation and blog retrieval. The summarization endpoint responded successfully to the request, though the short sample text did not produce a useful summary. A longer paragraph should be sent to test summary quality.